

Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Solutions: Exercise 1: Central Limit Theorem

The central limit theorem states that the sum or average of random samples from any probability distribution with a mean and standard deviation smaller than infinity will approximate a normal distribution, with a sufficient number of samples. This exercise should help you getting an intuition what a “sufficient number” of samples is and what this implies for the assumption of the normal distribution in experimental settings.

Your task is to:

1. Choose any - ideally asymmetric or totally random (e.g. using the `sample` function) - probability distribution
2. Compute the average of 3, 5, 10, and 20 samples from the distribution for 200 times.
3. Plot these averages in a histogram and try overlaying the density of the normal distribution with the mean and standard deviation of the plotted values

```
# empty environment
rm(list = ls())

# load libraries
library(bmm)
library(tidyverse)
```

We first define the elements from which we sample (1 to 5). Then we create a vector of probabilities that sums to one to determine the probability with which each element is sampled (here element 1 to 5). We generated random values from a uniform distribution and used the softmax function that normalizes a vector of n values into a probability vector of n values. Then we sample a given number of values from these elements.

```
sample_elements <- 1:5

my_probs <- bmm::softmax(runif(length(sample_elements)))
my_samples <- sample(sample_elements, size = 3, replace = T, prob = my_probs)
```

Then we can calculate the average for these sampled values.

```
average = mean(my_samples)
```

Now, we have to repeat this for 200 times.

```
# numer of repetitions
n_rep <- 200

# create dataframe to store simulation results
sim_results <- data.frame(
  sim_run = 1:n_rep,
  avg_sample2 = numeric(n_rep),
  avg_sample5 = numeric(n_rep),
  avg_sample10 = numeric(n_rep),
```

```

  avg_sample20 = numeric(n_rep)
)

# calculate the mean 200 times with 3, 5, 10, and 20 samples
for (i in 1:200) {
  my_samples <- sample(sample_elements, size = 20, replace = T, prob = my_probs)

  sim_results$avg_sample2[i] <- mean(my_samples[1:2])
  sim_results$avg_sample5[i] <- mean(my_samples[1:5])
  sim_results$avg_sample10[i] <- mean(my_samples[1:10])
  sim_results$avg_sample20[i] <- mean(my_samples[1:20])
}

# show first simulation results
head(sim_results)

##   sim_run avg_sample2 avg_sample5 avg_sample10 avg_sample20
## 1      1         3.5         3.0         2.8         3.00
## 2      2         2.0         3.0         3.1         3.00
## 3      3         5.0         3.8         3.8         3.40
## 4      4         2.0         2.2         2.5         2.75
## 5      5         2.0         1.8         2.4         2.55
## 6      6         5.0         4.0         2.7         2.70

```

Let's reshape the simulated data to make plotting a little easier, and the plot histograms. We overlaid the normal distribution for each histogram.

```

# format simulation results to long format
df_results_plot <- sim_results %>%
  pivot_longer(cols = starts_with("avg_")) %>%
  mutate(sample_size = str_split_i(name, "_", 2))

# calculate the overall mean and sd for samples with 2, 5, 10, and 20 samples
average_stat_sim <- df_results_plot %>%

  # calculate mean and sd for each sample size samples
  summarise(
    mean = mean(value),
    sd = sd(value),
    .by = "sample_size"
  ) %>%

  # create normal distribution with mean/sd for each sample size
  rowwise() %>%
  mutate(data = list(tibble(
    x = seq(mean - 4 * sd, mean + 4 * sd, length.out = 100),
    y = dnorm(x, mean = mean, sd = sd),
    sample_size = sample_size
  ))) %>%
  select(data) %>%
  unnest(data)

# plot histograms with overlaid normal distributions
ggplot(data = df_results_plot,
  aes(x = value)) +

```

```
facet_wrap(vars(sample_size)) +  
geom_histogram(aes(y = after_stat(density)), bins = 30) +  
geom_line(data = average_stat_sim, aes(x = x, y = y), col = "red")
```

